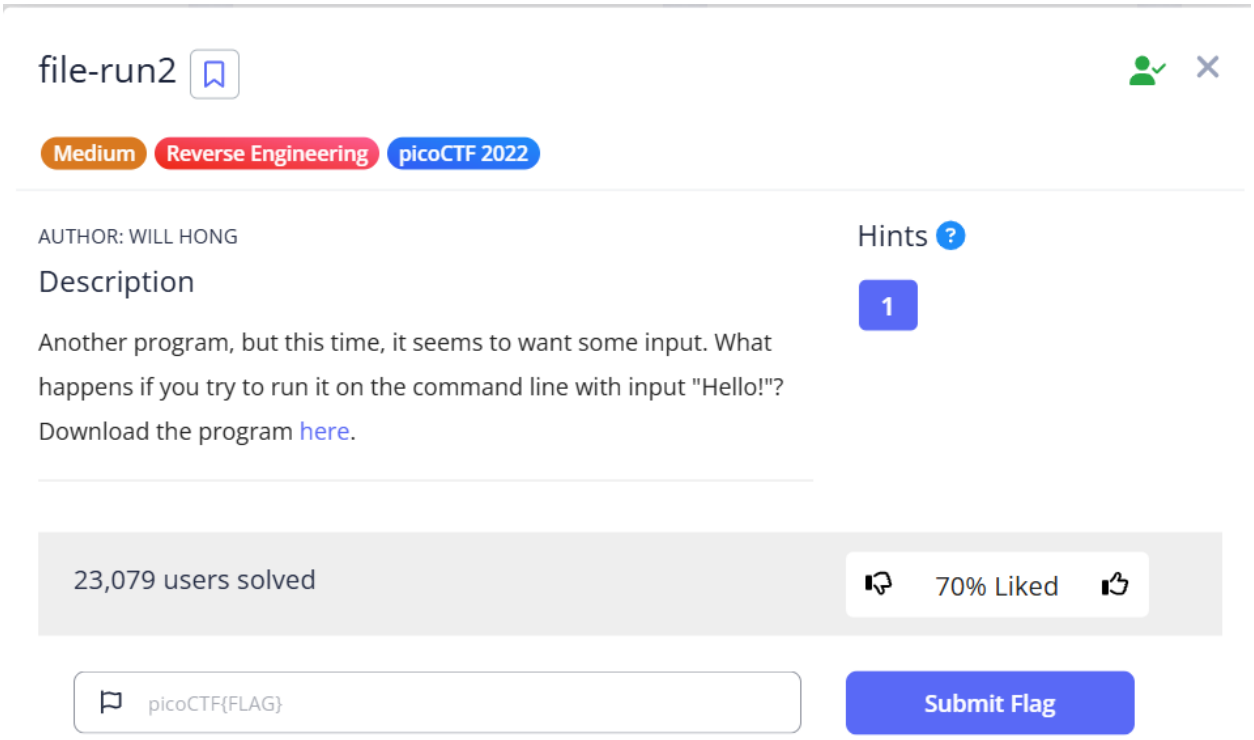
Saint Columban College
College of Computing Studies
Pagadian City

# File-Run2




**Author:** The Analyst: Hyposelenia

**Challenge:** File-Run2

**Category:** Reverse Engineering

**Date:** 03/27/26

I. Objective

The objective of this challenge is to understand how programs require **command-line arguments** and how providing the correct input allows the program to execute properly.

The goal is to supply the required argument to the program in order to **retrieve the flag**.

II. Background

In Linux, some programs are designed to accept **arguments** when they are executed.

An argument is:

Extra input given to a program during execution

Programs may depend on arguments to:

- control execution
- receive user input
- unlock certain behaviors

If the required argument is not provided, the program may not run correctly.

III. Tool Used

- **Kali Linux (Oracle VirtualBox)** - Used as the working environment
- **Linux Terminal** - Used to execute commands and run the program

IV. Methodology

1. Launched **Kali Linux** via Oracle VirtualBox
2. Downloaded the file using:

***wget <link_address>***

```
┌──(kali㉿kali)-[~/Downloads]
└─$ wget https://artifacts.picoctf.net/c/218/run
--2026-03-26 05:18:55--  https://artifacts.picoctf.net/c/218/run
Resolving artifacts.picoctf.net (artifacts.picoctf.net)... 18.172.2
Connecting to artifacts.picoctf.net (artifacts.picoctf.net)|18.172.
HTTP request sent, awaiting response... 200 OK
Length: 16736 (16K) [application/octet-stream]
Saving to: 'run'

run                                                 100%[===

2026-03-26 05:18:56 (652 KB/s) - 'run' saved [16736/16736]
```

3. Made the file executable:

***chmod +x <filename>***

```
┌──(kali㉿kali)-[~/Downloads]
└─$ ls
cat.jpg  leak  leak.tar  lyric-reader.py  red.png  run  unknown.zip
┌──(kali㉿kali)-[~/Downloads]
└─$ chmod +x run
┌──(kali㉿kali)-[~/Downloads]
└─$ ls
cat.jpg  leak  leak.tar  lyric-reader.py  red.png  run  unknown.zip
```

4. Executed the file with an argument:

***./<filename> "Hello!"***

```
┌──(kali㉿kali)-[~/Downloads]
└─$ ./run 'Hello!'
The flag is: picoCTF{F1r57_4rgum3n7_f65ed63e}
```

5. The program accepted the argument and displayed the **flag**

## V. Result

**picoCTF{F1r57_4rgum3n7_f65ed63e}**

```
The flag is: picoCTF{F1r57_4rgum3n7_f65ed63e}
```

VI. Explanation

**What is an Argument?**

An argument is:
A value you give to a program when you run it

Example:
***./program Hello***

- ./program → the program
- Hello → the argument

Think of it like:

- Program = a **machine** ⚙️
- Argument = the **input you give to the machine**

**Why did we add "Hello!"?**

The program expects at least **one argument**.

If you run:
***./<filename>***

It will not work properly because:

- No input was given

When you run:
***./<filename> "Hello!"***

Now:

- "Hello!" becomes the input
- The program continues running
- The flag is revealed

Most C programs use this structure:
***int main(int argc, char **argv)***

**Breakdown:**

- argc → number of arguments
- argv → list of arguments

**Example:**
***./program Hello***

- argc = 2
- argv[0] = program name
- argv[1] = "Hello"

The program likely checks:
"Did the user give me an argument?"
***If YES → continue***
***If NO → stop***

## VII. Conclusion

This challenge demonstrates how important it is to understand how programs accept and process **command-line arguments**.

By providing the required input, the program executed correctly and revealed the flag.

This reinforces a key concept in reverse engineering:

**Understanding how a program expects input is essential for interacting with and analyzing its behavior.**

**— The Analyst: Hyposelenia**